

Slovak University of Technology in Bratislava
Faculty of Informatics and Information Technologies

FIIT-XXXX-XXXXX

Bc. Richard Čerňanský

**Deep Learning Methods for Perception and
Prediction in Autonomous Systems**

Master's Thesis

Thesis Supervisor: Ing. Matej Halinkovič, PhD.

May 20xx

Slovak University of Technology in Bratislava
Faculty of Informatics and Information Technologies

FIIT-XXXX-XXXXX

Bc. Richard Čerňanský

**Deep Learning Methods for Perception and
Prediction in Autonomous Systems**

Master's Thesis

Study Programme:	Intelligent Software Systems
Study Field:	Computer Science
Training Workplace:	Bratislava, Slovakia
Thesis Supervisor:	Ing. Matej Halinkovič, PhD.

May 20xx

Čestne vyhlasujem, že som túto prácu vypracoval samostatne, na základe konzultácií a s použitím uvedenej literatúry.

I declare in my honor that I have prepared this work independently, on the basis of consultations and using the mentioned literature.

In Bratislava, May 20xx

Bc. Richard Čerňanský

Annotation

Slovak University of Technology in Bratislava
Faculty of Informatics and Information Technologies
Study Programme: Intelligent Software Systems
Study Field: Computer Science

Author: Bc. Richard Čerňanský

Master's Thesis: Deep Learning Methods for Perception and Prediction in Autonomous Systems

Thesis Supervisor: Ing. Matej Halinkovič, PhD.

May 20xx

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Keywords: 3-5 keywords

Anotácia

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií
Študijný program: Inteligentné softvérové systémy
Študijný odbor: Informatika

Autor: Bc. Richard Čerňanský

Diplomová práca: Deep Learning Methods for Perception and Prediction in Autonomous Systems

Vedúci diplomového projektu: Ing. Matej Halinkovič, PhD.

Máj 20xx

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Oblasť problematiky: 3-5 kľúčových slov

BP2, DP3: sem vložte finalne a podpísane zadanie: PDF z AISu

BP1, DP1, DP2: sem vložte návrh zadania: PDF z YonBanu

Acknowledgements / Podakovanie

The acknowledgments and the people to thank go here, don't forget to include your project advisor. . .

I also thank my family and friends who supported me during the writing of this thesis.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Formulation	2
1.2.1	Data Fusion	3
1.2.2	Dynamic Environments	3
1.2.3	Interaction Modeling	3
1.2.4	Real-World Constraints	3
1.3	Thesis Outline	4
2	Trajectory Prediction: Background and State of the Art	5
2.1	Taxonomy of Trajectory Prediction Methods	5
2.1.1	Physics-Based Methods	5
2.1.2	Learning-based methods	6
2.1.2.1	Classical Machine Learning Methods	6
2.1.2.2	Reinforcement Learning Methods	6
2.1.2.3	Deep Learning Methods	6
2.2	Prediction Paradigms	8
2.2.1	Modular (Detect-Track-Predict)	8
2.2.2	Joint Learning (End-to-End Perception & Prediction)	8
2.3	Datasets	9
2.4	Evaluation Metrics	9
2.5	Related Work	10
2.6	Summary and Starting Points	10
3	Objectives and Methodology	13
3.1	Objectives	13
3.2	Methodology	13
4	Design / Methods Description	15

5	Implementation	17
6	Experiments and Evaluation	19
6.1	Setup	19
6.2	Case Study - Experiment 01	20
6.3	Case Study - Experiment 02	20
6.4	Case Study - Experiment 03	20
6.5	Summary	20
7	Conclusion	21
7.1	Summary	21
7.2	Future Work	21
	Resumé	21
	List of Abbreviations	23
	List of Figures	26
	List of Tables	27
	References	31
A	Description of Digital Submission	B-1
B	Technical Documentation	B-3
B.1	Installation Manual	B-3
B.2	User Manual / Používateľská príručka	B-3
B.3	Developer Manual	B-3
C	Work Schedule	B-1
C.1	Work Schedule for BP1/DP1	B-1
C.2	Work Schedule for BP2/DP2	B-2
C.3	Work Schedule for DP3	B-2

Chapter 1

Introduction

In recent years, deep learning and computer vision methods have made significant advancements in the analysis of dynamic scenes and their applications in autonomous systems. The development of fields such as autonomous vehicles and robotics depends on solving tasks like object detection, object tracking, segmentation, and trajectory prediction. Specifically, autonomous vehicles (AVs) revolutionize transportation by saving drivers' time, improving safety and possibly mitigating congestion on the roads. Current autonomous driving systems consist of multiple sequential tasks such as perception, tracking, prediction, planning and control [1]. Perception task is responsible for detecting and understanding the objects within the environment including their categorization, location and size estimation. The tracking objective uses the detections from perception to find the relationships between objects across the frames in scene. The prediction modules then process this tracking information in order to forecast future positions. The predicted positions enter the planning stage that generates the most feasible and optimal path in terms of safety, energy consumption and following traffic rules. Last but not least, controlling task then prepares actuator commands which execute the planned motion.

1.1 Motivation

Real traffic is highly dynamic and unpredictable: other drivers may behave erratically, and sensors provide noisy or incomplete data. Diverse maneuvers, complex interactions, and inherent errors in sensory information make prediction challenging, yet solving these challenges is essential for having trustworthy AVs. In addition, autonomous systems operate under strict constraints on latency and reliability [2]. Predictions must be computed in real time on embedded hardware and remain accurate over long hori-

zons to be useful for driving in real-life scenarios. Industry recognizes the need for fast execution, uncertainty handling, and generalization in prediction models to ensure robust operation in varied conditions [3, 4, 5]. Reliable trajectory prediction is therefore critical for safe autonomous driving and motivates intensive research into methods that can forecast the multi-agent future with practical efficiency. For these reasons, this thesis focuses on trajectory prediction, exploring how current state-of-the-art models could be modified or extended to achieve better accuracy and efficiency.

1.2 Problem Formulation

Trajectory prediction is the task of forecasting the future motion of the traffic participants that surround the ego vehicle. Its input is not a trajectory in the purely geometric sense, but rather the past states of these participants as they are captured in the environmental information collected by sensors, such as camera images, LiDAR point clouds, radar returns or HD maps providing lane geometry and traffic-rule priors. However, in this thesis only camera and LiDAR is employed. A model that relies on a single source is unimodal, whereas a model that exploits several of them at once is considered multi-modal. In the latter case the measurements are heterogeneous in dimensionality, sampling rate and coordinate frame, so the features extracted from the individual sources have to be fused into a unified representation of the scene before any forecasting takes place [6]. The output is the forecasted trajectory, that is, a sequence of future states of the observed agents over a fixed prediction horizon.

Formally, trajectory prediction can be stated as the problem of constructing a mapping Φ from the observed evidence $\mathcal{I}$ about the scene to the future states $\mathcal{O}$ of the traffic participants, from the current time up to a prediction horizon T [7]:

$$\begin{aligned}\Phi : \mathcal{I} &\rightarrow \mathcal{O}, \\ \mathcal{O} &= \{S^0, S^1, \dots, S^T\},\end{aligned}\tag{1.1}$$

where S^0 is the current state and S^1 to S^T are the states at the individual future time steps, each of them collecting the states of all N observed participants. Depending on the available modalities and on the downstream task, a single state can take different forms, for example

$$S^t = \{x^t, y^t\} \quad \text{or} \quad S^t = \{x^t, y^t, l^t, w^t, \theta^t\},\tag{1.2}$$

where x^t and y^t are the coordinates of the centre point of the object, l^t and w^t its length and width, and θ^t its orientation [7]. In trajectory prediction the state is usually reduced to the planar position, so that $\mathcal{O}$ corresponds to the set of forecasted trajectories of the

participants in the scene. Since the future of a traffic scene is inherently uncertain, the mapping in (1.1) is in practice often extended to output several plausible trajectories together with their confidences, allowing the planning stage to reason over the alternatives instead of committing to a single hypothesis [2]. The following subsections summarise the main challenges that this formulation raises.

1.2.1 Data Fusion

The information describing the scene comes from sources that are complementary but incompatible. Cameras provide dense semantic detail without depth and degrade at night, in glare or in poor lighting conditions, LiDAR provides accurate geometry but suffers in rain etc. These sources can be combined at the level of raw measurements (early), of learned features (intermediate) or of the individual predictions (late). Since every added modality costs latency and introduces noise the network is prone to overloading with parameters or can overspecialize on the most informative source and lose accuracy once the weather or illumination degrades it [6].

1.2.2 Dynamic Environments

Traffic is nonstationary, as density, weather, illumination and road layout change, so that the conditions met during deployment need not resemble those seen during development. The future is moreover not unique, since the same observed history is usually compatible with several maneuvers, and this aleatoric uncertainty originated in the erratic decisions of other drivers does not diminish with additional data. Therefore the task requires a set of socially compliant trajectories produced in time for the planning stage [6].

1.2.3 Interaction Modeling

The motions of traffic participants are not independent and traffic signals constrain all of them, so that the joint future does not split into separate per-agent distributions. Capturing the agent to agent and the agent to map relations requires representations able to operate on a variable and unordered number of entities, which is why structured formulations such as graphs and attention mechanisms prevail. Where these relations are modelled poorly, forecasts that appear plausible for each agent in isolation become inconsistent at the level of the scene comprising of multiple entities [2].

1.2.4 Real-World Constraints

Prediction is one stage of a control loop closing at a fixed rate, which leaves it a budget of tens of milliseconds on embedded hardware limited in memory and power. Its cost

grows with the number of participants and becomes quadratic once their interactions are modelled, so the least favourable case is dense traffic, precisely where prediction matters most. Model capacity therefore has to be weighed against computational efficiency, as accuracy alone does not make a model deployable [4, 2].

1.3 Thesis Outline

Chapter 2

Trajectory Prediction: Background and State of the Art

concrete header for the given thesis's theme!

2.1 Taxonomy of Trajectory Prediction Methods

In the literature, there are numerous different prediction methods, and several criteria enable their systematic classification, such as the modality and the representation of the input, the form of the produced output, or the paradigm by which prediction is integrated with perception. Here the modeling methods are classified according to the categorization presented in [6].

2.1.1 Physics-Based Methods

These rely on kinematic or dynamic models of motion, using physical equations to extrapolate future positions and not accounting for uncertainty. Examples include constant-velocity, constant-acceleration models (Single-Trajectory simulation) [8] [9]. Kalman filters address the uncertainty limitation of the previous models by modeling the vehicle's state as a Gaussian distribution. It predicts trajectories as a two step process of filtering (denoising current state from noisy measurements) and forecasting [10]. Monte Carlo methods, in particular particle filtering, drop the linearity and Gaussianity assumptions of the Kalman filter by approximating the state distribution with a set of weighted samples propagated through the motion model, which permits arbitrarily shaped and multimodal distributions at the cost of a higher computational load [11]. Physics-based approaches are simple and fast, but they struggle with complex maneuvers or

sudden behavior changes since they lack contextual understanding. Some variants combine multiple motion models to handle different maneuvers (Multiple-Model methods) [12].

2.1.2 Learning-based methods

Unlike physics-based, learning-based methods learn the motion dynamics from data. Here, three groups of approaches are mentioned.

2.1.2.1 Classical Machine Learning Methods

These methods learn motion patterns from data using statistical models, but without deep neural networks. Common approaches include hidden Markov models (HMMs) [13], Gaussian processes (GPs) based prediction models [14], dynamic Bayesian networks (DBNs) [15], and Gaussian mixture models (GMMs) [16]. They can capture uncertainty to a degree (e.g., an HMM can model discrete maneuver states, GPs provide a distribution over trajectories) and were popular before deep learning. However, their capacity to model very complex, high-dimensional scenarios is limited compared to modern deep nets.

2.1.2.2 Reinforcement Learning Methods

Reinforcement learning (RL) methods treat prediction as a sequential decision problem. Here, an agent learns a policy (via reward feedback) to mimic human driving behavior, effectively planning likely trajectories. Since the reward that drives human driving is not known, inverse reinforcement learning (IRL) recovers it from expert demonstrations and uses it to regularise the predictor [17], while imitation learning (IL) skips the reward altogether and extracts the policy directly from the observed trajectories [18]. RL-based predictors can naturally handle multi-agent interactions by modeling how agents might react to each other, and can ensure physically feasible outputs through learned policy constraints. However, RL methods can be data-hungry, harder to train and stabilize, and historically have been more common in planning than in pure trajectory forecasting. They are an active research area for capturing game-theoretic interactions and long-horizon planning in prediction.

2.1.2.3 Deep Learning Methods

Various Deep Learning methods perform successfully in learning motion dynamics and now dominate trajectory prediction. Their strengths and limitations differ mainly according to their architectural design.

Recurrent Neural Networks

Recurrent neural networks (RNNs) and long short-term memory (LSTM) capture temporal history via hidden states updated across timesteps. They are able to effectively capture temporal patterns for trajectory prediction task. Historically, single recurrent models were used to map past trajectories to future paths [19], however recent works facilitate groups or stacks of multiple models [20] or employ them for one of the sub-tasks like detection [21]. They converge reliably and can be trained autoregressively, in the same mode in which they are later run at inference, but they lack spatial reasoning and struggle with long-range temporal dependencies [22].

Convolutional Neural Networks

Models built on convolutional neural networks (CNNs) are mostly responsible for feature extraction such as capturing objects' positions and environmental context [23]. Temporal convolutional networks (TCNs) are able to model temporal dependencies through the causal convolution layers that preserve the temporal order [24]. They extract spatial features from image-like inputs such as bird's-eye view (BEV) or rasterised maps efficiently and permit fast inference, although the fixed grid on which they operate represents the individual agents only implicitly and is more expensive than a vectorised encoding of the same scene [22].

Graph Neural Networks

Graph neural networks (GNNs) represent the scene as a graph whose nodes are the agents and map elements and whose edges encode the relations between them, which makes them a natural fit for modelling spatial and temporal interactions [25]. They excel in dense interaction scenarios by modelling agent-to-agent relations explicitly, but they add unnecessary complexity in sparse environments such as highway driving, and their dynamic-graph variants rely on consistent object identities and therefore inherit the errors of the tracker [6].

Transformers

Transformers replace recurrence with self-attention, which relates every agent and every map element to all the others in a single step and thus captures long-range dependencies without having to propagate information through a hidden state [26]. Since attending over all agents, time steps and map elements at once is expensive, practical models factorise the attention along these axes and decode the future with a set of learned queries that produce several trajectories in parallel [4]. Their capacity for long-term dependencies makes them particularly promising for long-horizon prediction, but they are highly sensitive to training stability and demand extensive tuning and warm-up [6]. Transformer systems pair a strong scene encoder with multi-modal decoders and explicit interaction and map context [26, 27].

2.2 Prediction Paradigms

In this section, the prediction paradigms are discussed. The prediction paradigm is an important characteristic of a model, as it determines how the driving stack is decomposed into modules, what is optimised during training, and what information is passed from perception to prediction. The advantages and the limitations of a given model largely follow from this choice.

2.2.1 Modular (Detect-Track-Predict)

This approach has been of the most common so far implemented in the literature. The paradigm consists of sequential steps of detection, tracking and prediction, which were independently under an extensive research. First, the sensor data is captured and detector localizes the objects present in the scene, usually as a set of bounding boxes with the corresponding class labels. The tracking module then associates these detections across consecutive frames, opening a tracklet for every newly observed object and preserving its identity as the scene evolves. Finally, the prediction module consumes the active tracklets and forecasts the future positions of the corresponding agents [6]. Even though this sequential decomposition is intuitive and each module can be developed and debugged separately, the stages are optimised in isolation which causes errors to propagate across modules and results in suboptimal performance due to the lack of end-to-end optimization [7].

TODO: add some examples of models

2.2.2 Joint Learning (End-to-End Perception & Prediction)

Joint learning methods treat detection and motion forecasting as a single optimization problem instead of two independent stages. These unified optimization goals help to mitigate the error accumulation in the intermediate steps. Such models are trained with a compound loss over both tasks. They differ in how tightly the stages are coupled, ranging from a shared light detection and ranging (LiDAR) backbone co-optimized through a combined loss [28], through BEV features reused by both tasks [29], to full gradient flow from the prediction loss back through the detector [30]. Query-based models take this integration further by representing every agent as a learnable query that is iteratively refined while attending to scene and map features, so that detection, tracking and forecasting are resolved together [31, 32]. The price of this coupling is that task boundaries blur, which makes poor performance hard to attribute to a specific component, and the iterative refinement of queries adds computational overhead that complicates real-time deployment in dense scenes [6].

2.3 Datasets

Research in trajectory prediction has been accelerated by large-scale autonomous driving datasets that provide rich multimodal sensor data and trajectory annotations. Some prominent datasets include:

- **KITTI (2012):** Early autonomous vehicle (AV) benchmark with stereo, LiDAR, and global positioning system (GPS)/inertial measurement unit (IMU), great for detection, tracking, and simultaneous localization and mapping (SLAM). It is small (150 scenes) and supports only short-horizon forecasting, so it's limited for learning social behavior. [33]
- **nuScenes (2020):** Large multimodal suite (6 cameras, 5 radars, 1 LiDAR, 360°) with 1,000 twenty-second scenes, 3D boxes, and high-definition maps (HD maps). It added a prediction challenge (≈ 6 s horizon) and is ideal for multi-sensor fusion and map-aware forecasting. [34]
- **Argoverse (2019):** Includes a Motion Forecasting set with 324k five-second scenarios and rich HD lane graphs (Miami/Pittsburgh). It popularized map-based prediction and uses ADE/FDE and MR; Argoverse 2 expands coverage. [35]
- **Waymo Open Motion (2021):** Very large-scale forecasting dataset (>100 k twenty-second scenarios at 10 Hz) with HD maps and many interactive scenes. It introduced mAP and MR for multi-modal sets and is a key benchmark for vectorized/transformer models. [3]
- **Lyft Level 5:** Large urban dataset with synchronized LiDAR/cameras and HD maps, similar in format to nuScenes. Widely used for 3D detection/tracking and baseline motion forecasting with map context. [36]
- **ApolloScape:** Chinese urban driving data covering detection, segmentation, tracking, and mapping. Useful for perception and localization; less standardized for forecasting than nuScenes/Argoverse. [37]

2.4 Evaluation Metrics

Across these benchmarks, common metrics to evaluate trajectory prediction [6, 3] include:

- **Average Displacement Error (ADE):** the average Euclidean distance between the predicted trajectory and the ground-truth trajectory over all time steps. It measures overall prediction accuracy.
- **Final Displacement Error (FDE):** the Euclidean distance between the predicted final position (at the prediction horizon, e.g., 3 s or 6 s) and the true final position. This emphasizes endpoint accuracy.
- **minADE and minFDE (multi-modal prediction):** for models that output K pos-

sible futures per agent, these take the error of the closest prediction to ground truth (assume the model's best hypothesis). minADE/FDE reflect the best-case performance of a probabilistic predictor.

- **Miss Rate (MR):** usually defined as the fraction of cases where none of the K predicted trajectories is within a certain distance (e.g., 2 m) of the ground truth at the final time. A lower miss rate means the model almost always has at least one prediction near the true outcome, which is important for safety.
- **Mean Average Precision (mAP):** introduced in Waymo's benchmark, mAP treats trajectory prediction as a detection problem in trajectory space. It evaluates the precision-recall curve of the model's predicted set of trajectories against ground truth, taking into account both spatial tolerance and assigned confidence scores. It effectively rewards methods that produce a set of diverse predictions that cover the ground truth while avoiding too many implausible guesses.
- **Other metrics:** Negative Log-Likelihood (NLL) for probabilistic methods (measuring the quality of the predicted probability distribution), and occasionally collision rate or overlap metrics to ensure predicted trajectories are physically feasible (not crashing into others or violating road constraints).

Using these metrics in combination provides a comprehensive evaluation: ADE/FDE for accuracy, minADE/minFDE and MR for multimodal coverage, and mAP for overall usefulness of the prediction set. The literature often reports all of them to compare methods. For instance, a method might achieve lower ADE than another (better average accuracy), but if its miss rate is high it means it sometimes totally misses possible outcomes, a trade-off the proposal's approach must consider.

2.5 Related Work

This part is important in determining the objectives of the Thesis as we will state in Section 2.6 and in Chapter 3.

2.6 Summary and Starting Points

Summary of Chapter 2 ... several paragraphs... 1/2 to 1 page

Based on the study presented in Chapter 2, the following gaps / needs are identified with potential to improve:

- question / gap / need
- ...
- question / gap / need

To answer these questions, the following starting points are proposed.

- Starting point 1:
- Starting point 2:
- Starting point 3:

Priebežná správa BP1/DP1 musí obsahovať:

1. Prehľad podobných riešení alebo produktov, s ktorými sa študent oboznámil pri štúdiu danej problematiky.
2. Špecifikáciu požiadaviek na vytvárané riešenie alebo produkt (Chapter 3).
3. Návrh
4. Vlastne zhodnotenie práce za príslušné obdobie (Chapter C - Section C.1).

Chapter 3

Objectives and Methodology

3.1 Objectives

Vybrané ciele pre navrhnuté riešenie alebo špecifikácia požiadaviek (funkčné a nefunkčné požiadavky) na naimplementovaný softvérový produkt.

- povinne pre DP a DiZP, ale aj BP môže
- max 2 pages for the whole chapter.

As presented in Chapter 2, the area of... is wide with many interesting unsolved problems. Therefore, the goals of this thesis are focused on selected objectives (maximum 3 objective) to... as follows:

- Objective 1:
- Objective 2:
- Objective 3:

3.2 Methodology

The following steps have been chosen for the design and development of ...

- Step 1:
- Step 2:
- ...
- Step n:

These steps are commitments to each other in the context of ... of the work.

Chapter 4

Design / Methods Description

Concrete header for the proposed design of the thesis's solution

This chapter can contain:

- Specification based on the Sections 2.6
- Design
- Technologies (can also be/moved in/to Chapter 5 Implementation)

Description how the design works (as much as possible from the below list)

- We need to separate our design from our concrete implementation
- Architecture, Block scheme, Event diagram, Sequence diagram, Algorithms, ...
- Text description of each blocks and their connections and/or dependencies
- Time complexity and memory complexity estimation for each execution step and the whole design. Communication complexity of the design (if any).

Figure 4.1 is about ... description of each blocks

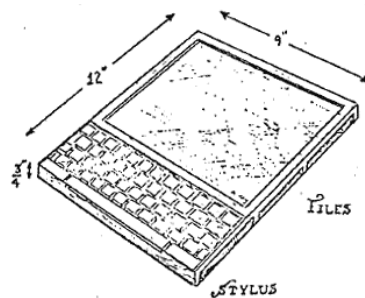


Figure 4.1: Architecture/Workflow

Chapter 5

Implementation

Concrete header for the implementation of the proposed design

The concrete realization of our proposed design

Concrete tools, software, solutions based on concrete dataset

Pseudocode of Algorithms, or Code snippets

$$\text{Bounce Rate} = \frac{\text{Single page sessions}}{\text{Total sessions}} \quad (5.1)$$

Chapter 6

Experiments and Evaluation

6.1 Setup

hardware,
software, libraries, versioning,
virtual environment description and setting,
datasets, NDA,
etc.

Machine	Remote	Google Colab	Kaggle	FIIT Cloud	Local
Python	3.9.19	3.10.12	-	-	3.10.12
PyTorch	-	-	-	-	-
TensorFlow	2.10	2.15	-	-	2.14
GPU (NVIDIA)	RTX xx	Tesla T4 15 GB	Tesla P100 16GB	-	-
CUDA	12.2	12.2	-	-	-
cuDNN	8.1.0	N/A	-	-	-
CUDA Toolkit	11.2.2	N/A	-	-	-
RAM	32/64 GB	51 GB	30 GB	32 GB	16 GB
CPU	8x cores Intel i7	-	Xeon 4x cores	4x cores	-
Storage	SSD	Google Driver	73 GB	128 GB	HDD

Table 6.1: Hardware, software and versioning used for realization

6.2 Case Study - Experiment 01

6.3 Case Study - Experiment 02

6.4 Case Study - Experiment 03

6.5 Summary

Main *key findings* of the work based on the Experiment results

- add ...
- ...

Discussion about various considerations based on the experiment results (if needed).

Regarding *limitations* of the work ...

- add ...
- ...

Chapter 7

Conclusion

7.1 Summary

Summary of the whole work including SOTA, design, implementations, experiments, evaluations, key findings ... 1 page

7.2 Future Work

1/2 page

Resumé

Chapter je bez cislovania, pouziva sa len **boldovanie textu** na zyzraznenie useku textu. Bez Figure, Table, atd. ak treba tak odvolava sa na originalne Figure a Table.

10% v slovenčine pre tých, ktorí majú prácu v EN: preklad

EN	SK
Introduction	primerane strucne
SOTA	primerane strucne
SOTA - cast Summary and Starting Points	cele
Objective and Methodology	cele
Design, Implementation	primerane strucne
Experiments and Evaluation	primerane strucne
Experiments and Evaluation - cast Summary	cele
Conclusion and Future Work	cele

Table 7.1: Resumé

List of Abbreviations

ADE	Average Displacement Error
AV	autonomous vehicle
AVs	autonomous vehicles
BEV	bird's-eye view
CNN	convolutional neural network
DBN	dynamic Bayesian network
FDE	Final Displacement Error
GMM	Gaussian mixture model
GNN	graph neural network
GP	Gaussian process
GPS	global positioning system
HD map	high-definition map
HMM	hidden Markov model
IL	imitation learning
IMU	inertial measurement unit
IRL	inverse reinforcement learning
LiDAR	light detection and ranging
LSTM	long short-term memory
mAP	mean Average Precision
MR	Miss Rate

NLL Negative Log-Likelihood

RL reinforcement learning

RNN recurrent neural network

SLAM simultaneous localization and mapping

TCN temporal convolutional network

List of Figures

4.1	Architecture/Workflow	15
A.1	The structure of the ZIP file	B-2

List of Tables

6.1	Hardware, software and versioning used for realization	19
7.1	Resumé	23

References

- [1] Li Chen et al. “End-to-end autonomous driving: Challenges and frontiers”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.12 (2024), pp. 10164–10183. DOI: 10.1109/TPAMI.2024.3435937.
- [2] Yanjun Huang et al. “A Survey on Trajectory-Prediction Methods for Autonomous Driving”. In: *IEEE Transactions on Intelligent Vehicles* 7.3 (2022), pp. 652–674. DOI: 10.1109/TIV.2022.3167103.
- [3] Scott Ettinger et al. “The Waymo Open Motion Dataset: A Large-Scale Benchmark for Motion Forecasting”. In: *Proceedings of the Conference on Robot Learning (CoRL)*. 2021.
- [4] Nigamaa Nayakanti et al. *Wayformer: Motion Forecasting via Simple & Efficient Attention Networks*. 2022. arXiv: 2207.05844 [cs.CV]. URL: <https://arxiv.org/abs/2207.05844>.
- [5] *Road vehicles — Safety of the intended functionality (SOTIF)*. International Organization for Standardization, 2022.
- [6] Nadya Abdel Madjid et al. “Trajectory Prediction for Autonomous Driving: Progress, Limitations, and Future Directions”. In: *arXiv preprint arXiv:2503.03262* (2025). URL: <https://arxiv.org/abs/2503.03262>.
- [7] Liang Yufan Hu, Qilong Kong, and Bin Fan. “A Survey on End-to-End Perception and Prediction for Autonomous Driving”. In: *Machine Intelligence Research* 22.6 (2025), pp. 999–1030. DOI: 10.1007/s11633-025-1558-0.
- [8] Chiu-Feng Lin, A. Galip Ulsoy, and David J. LeBlanc. “Vehicle Dynamics and External Disturbance Estimation for Vehicle Path Prediction”. In: *IEEE Transactions on Control Systems Technology* 8 (2000), pp. 508–518. URL: <https://api.semanticscholar.org/CorpusID:33770615>.
- [9] Rick Miller and Qingfeng Huang. “An Adaptive Peer-to-Peer Collision Warning System”. In: *IEEE 55th Vehicular Technology Conference (VTC Spring)*. Vol. 1. 2002, pp. 317–321. URL: <https://api.semanticscholar.org/CorpusID:18526419>.

- [10] Rudolf E. Kalman. "A New Approach to Linear Filtering and Prediction Problems". In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45. doi: 10.1115/1.3662552.
- [11] Neil J. Gordon, David J. Salmond, and Adrian F. M. Smith. "Novel approach to nonlinear/non-Gaussian Bayesian state estimation". In: *IEE Proceedings F (Radar and Signal Processing)* 140.2 (1993), pp. 107–113. doi: 10.1049/ip-f-2.1993.0015.
- [12] X. Rong Li and Vesselin P. Jilkov. "Survey of Maneuvering Target Tracking. Part V: Multiple-Model Methods". In: *IEEE Transactions on Aerospace and Electronic Systems* 41 (2005), pp. 1255–1321. doi: 10.1109/TAES.2005.1561886.
- [13] Nachiket Deo, Akshay Rangesh, and Mohan M. Trivedi. "How Would Surround Vehicles Move? A Unified Framework for Maneuver Classification and Motion Prediction". In: *IEEE Transactions on Intelligent Vehicles* 3 (2018), pp. 129–140. url: <https://api.semanticscholar.org/CorpusID:9228629>.
- [14] Jack M. Wang, David J. Fleet, and Aaron Hertzmann. "Gaussian Process Dynamical Models for Human Motion". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 30 (2008), pp. 283–298. doi: 10.1109/TPAMI.2007.1167.
- [15] Yuxiao Li et al. "Pedestrian Trajectory Prediction Combining Probabilistic Reasoning and Sequence Learning". In: *IEEE Transactions on Intelligent Vehicles* 5 (2020), pp. 461–474. doi: 10.1109/TIV.2020.2966117.
- [16] Jürgen Wiest et al. "Probabilistic Trajectory Prediction with Gaussian Mixture Models". In: *2012 IEEE Intelligent Vehicles Symposium*. 2012, pp. 141–146. doi: 10.1109/IVS.2012.6232277.
- [17] Doseop Choi, Kyoungwook Min, and Jeongdan Choi. "Regularising Neural Networks for Future Trajectory Prediction via Inverse Reinforcement Learning Framework". In: *IET Computer Vision* 14 (2020), pp. 192–200. doi: 10.1049/iet-cvi.2019.0546.
- [18] Mengshi Qi et al. "Imitative Non-Autoregressive Modeling for Trajectory Forecasting and Imputation". In: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 12733–12742. doi: 10.1109/CVPR42600.2020.01275.
- [19] M. Hoy et al. "Learning to Predict Pedestrian Intention via Variational Tracking Networks". In: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. 2018, pp. 3132–3137. doi: 10.1109/ITSC.2018.8569641.
- [20] L. Xin et al. "Intention-Aware Long Horizon Trajectory Prediction of Surrounding Vehicles Using Dual LSTM Networks". In: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE Press, 2018, pp. 1441–1446.
- [21] Hao Shao et al. "ReasonNet: End-to-End Driving with Temporal and Global Reasoning". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023.

- [22] Jianbang Liu et al. “A Survey on Deep-Learning Approaches for Vehicle Trajectory Prediction in Autonomous Driving”. In: *2021 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. 2021. doi: 10.1109/ROBIO54168.2021.9739407.
- [23] Tung Phan-Minh et al. “CoverNet: Multimodal Behavior Prediction Using Trajectory Sets”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 14062–14071. doi: 10.1109/CVPR42600.2020.01408.
- [24] N. Nikhil and B. T. Morris. “Convolutional Neural Network for Trajectory Prediction”. In: *Computer Vision – ECCV 2018 Workshops*. Ed. by L. Leal-Taixé and S. Roth. Cham: Springer International Publishing, 2019, pp. 186–196.
- [25] C. Wang, S. Cai, and G. S. H. Tan. “GraphTCN: Spatio-Temporal Interaction Modeling for Human Trajectory Prediction”. In: *2021 IEEE Winter Conference on Applications of Computer Vision (WACV)*. 2021, pp. 3449–3458. url: <https://api.semanticscholar.org/CorpusID:212725547>.
- [26] Yicheng Liu et al. “Multimodal Motion Prediction with Stacked Transformers”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, pp. 7573–7582. doi: 10.1109/CVPR46437.2021.00749.
- [27] Jiquan Ngiam et al. “Scene Transformer: A Unified Architecture for Predicting Multiple Agent Trajectories”. In: *International Conference on Learning Representations (ICLR)*. 2022. url: https://openreview.net/forum?id=KJk%5C_H1To6A.
- [28] Wenjie Luo, Bin Yang, and Raquel Urtasun. “Fast and Furious: Real Time End-to-End 3D Detection, Tracking and Motion Forecasting with a Single Convolutional Net”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 3569–3577. doi: 10.1109/CVPR.2018.00376.
- [29] Anthony Hu et al. “FIERY: Future Instance Prediction in Bird’s-Eye View from Surround Monocular Cameras”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 15273–15282.
- [30] Ming Liang et al. “PnPNet: End-to-End Perception and Prediction with Tracking in the Loop”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 11553–11562. doi: 10.1109/CVPR42600.2020.01157.
- [31] Junru Gu et al. “ViP3D: End-to-End Visual Trajectory Prediction via 3D Agent Queries”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023, pp. 5496–5506.
- [32] Sergio Casas et al. “DeTra: A Unified Model for Object Detection and Trajectory Forecasting”. In: *European Conference on Computer Vision (ECCV)*. 2024. url: <https://api.semanticscholar.org/CorpusID:270357489>.
- [33] Andreas Geiger, Philip Lenz, and Raquel Urtasun. “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2012, pp. 3354–3361.

-
- [34] Holger Caesar et al. “nuScenes: A Multimodal Dataset for Autonomous Driving”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020.
 - [35] Ming-Fang Chang et al. “Argoverse: 3D Tracking and Forecasting With Rich Maps”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.
 - [36] R. Kesten et al. “Level 5 AV Dataset 2019”. In: *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2019.
 - [37] Xiaoyong Huang et al. “ApolloScape: 3D Car Understanding from Monocular Images”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2018.

Appendix A

Description of Digital Submission

Thesis Evidence Number:

FIIT-XXXX-XXXXX

Obsah elektronického média je väčší ako 1GB a preto je odovzdaný vedúcemu záverečnej práce (doc. Ing. Giang Nguyen Thu, PhD.).

Name of submitted archive: DP_MenoPriezvisko.zip

The digital part included in the thesis contains the following files:

- Ubuntu OS: run in command line \$ tree to get a directory structure as below.
- Inspiration from <https://github.com/FIIT-ISA/cookiecutter-data-science>

ROOT // Root directory.

```
|
|— flatbuffers // This flatbuffers director
|— scheme
```

Appendix A. Description of Digital Submission



LICENSE	
Makefile	<- Makefile with commands like `make data` or `make train`
README.md	<- The top-level README for developers using this project.
data	
external	<- Data from third party sources.
interim	<- Intermediate data that has been transformed.
processed	<- The final, canonical data sets for modeling.
raw	<- The original, immutable data dump.
docs	<- A default Sphinx project; see sphinx-doc.org for details
models	<- Trained and serialized models, model predictions, or model summaries
notebooks	<- Jupyter notebooks. Naming convention is a number (for ordering), the creator's initials, and a short `-` delimited description, e.g. `1.0-jqp-initial-data-exploration`.
references	<- Data dictionaries, manuals, and all other explanatory materials.
reports	<- Generated analysis as HTML, PDF, LaTeX, etc.
figures	<- Generated graphics and figures to be used in reporting
requirements.txt	<- The requirements file for reproducing the analysis environment, e.g. generated with `pip freeze > requirements.txt`
setup.py	<- makes project pip installable (pip install -e .) so src can be imported
src	<- Source code for use in this project.
__init__.py	<- Makes src a Python module
data	<- Scripts to download or generate data
make_dataset.py	
features	<- Scripts to turn raw data into features for modeling
build_features.py	
models	<- Scripts to train models and then use trained models to make predictions
predict_model.py	
train_model.py	
visualization	<- Scripts to create exploratory and results oriented visualizations

Figure A.1: The structure of the ZIP file

Appendix B

Technical Documentation

B.1 Installation Manual

1. Clone the project from the Git repository (URL) OR unzip the ZIP file ...
2. Run `pip install -r requirements.txt` in the root folder of the project
3. Run `python -m spacy download en` to have the data corpus ...

B.2 User Manual / Používateľská príručka

1. Spustenie programu funguje pomocou hlavného Python súboru *main.py* príkazom `python main.py`.
2. Po spustení programu sa zobrazí grafické rozhranie programu v ktorom sa dá hneď na hlavnej obrazovke spustiť sumarizácia pre jeden vstupný súbor.
3. Tlačidlami v hornej časti obrazovky sa dá presúvať medzi funkcionalitami programu ako je viac súborová sumarizácia alebo zobrazovanie článkov. Na týchto obrazovkách sa tlačidlom *Back to main menu* dá vrátiť späť na hlavnú obrazovku.

Screenshot(s) používateľského grafického rozhrania (GUI) ako obrázok (Figure)

(Another variant without GUI:) The main objective of this work is to ... Several case studies were presented that examine the capabilities of ... to show the feasibility and achievability of ... Users in our work are considered to work directly with the code as described in Section B.1.

B.3 Developer Manual

if needed then add ...

Appendix C

Work Schedule

C.1 Work Schedule for BP1/DP1

Week	Plan
1. - 2.	...
3. - 4.	...
5. - 6.	...
7. - 8.	...
9. - 10.	...
11. - 12.	...

Some explanation text... (if needed) ... a few sentences

Self-Evaluation of Work Schedule for BP1/DP1: Vyjadrenie k plneniu plánu: some text ... one paragraph ... self-evaluation

(In the best case:) The BP/DP 1 work plan is considered satisfied and successful.

C.2 Work Schedule for BP2/DP2

Week	Plan
1. - 2.	...
3. - 4.	...
5. - 6.	...
7. - 8.	...
9. - 10.	...
11. - 12.	...

Some explanation text... (if needed) ... a few sentences

Self-Evaluation of Work Schedule for BP2/DP2: Vyjadrenie k plneniu plánu: some text ... one paragraph ... self-evaluation

(In the best case:) The BP/DP 2 work plan is considered satisfied and successful.

C.3 Work Schedule for DP3

Week	Plan
1. - 2.	...
3. - 4.	...
5. - 6.	...
7. - 8.	...
9. - 10.	...
11. - 12.	...

Some explanation text... (if needed) ... a few sentences

Self-Evaluation of Work Schedule for DP3: Vyjadrenie k plneniu plánu: some text ... one paragraph ... self-evaluation

(In the best case:) The DP 3 work plan is considered satisfied and successful.